



Programación multihilo en Java - Cena de los Filósofos

ÍNDICE

1. Introducción.....	1
○ Descripción del problema de la cena de los filósofos.....	1
○ Objetivos de la práctica y relación con la programación multihilo.....	1
2. Diseño de la solución.....	2
○ Explicación de la arquitectura general.....	2
○ Descripción de las clases principales y sus responsabilidades.....	2
○ Decisiones de diseño sobre el uso de monitores y/o semáforos.....	2
3. Implementación concurrente.....	3
○ Detalle del funcionamiento de MonitorFilosofos.....	3
○ Descripción de las variantes con semáforos y portero.....	4
○ Justificación de cómo se evita el deadlock y cómo se reduce la starvation.....	4
4. Pruebas y resultados.....	5
○ Descripción de las pruebas realizadas.....	5
○ Resumen de las pruebas unitarias implementadas con JUnit.....	5
○ Selección de capturas de pantalla del depurador.....	6
5. Conclusiones y trabajo futuro.....	8
○ Resumen de lo aprendido sobre programación multihilo en Java.....	8
○ Posibles mejoras o extensiones.....	8

1. Introducción

- **Descripción del problema de la cena de los filósofos.**

El Problema de la **Cena de los Filósofos** es un escenario de concurrencia donde **N filósofos comparten N tenedores en círculo**. Para comer, **cada filósofo necesita dos tenedores**, el de su izquierda y el de su derecha. El reto de este problema es conseguir diseñar un **protocolo** que nos permita que todos los filósofos **puedan comer lo suficiente**, sin caer en problemas como el interbloqueo (*deadlock*), inanición (*starvation*) o condiciones de carrera (*race conditions*).

- **Objetivos de la práctica y relación con la programación multihilo.**

Los objetivos de la práctica se resumen en diseñar un sistema que gestione el acceso a recursos compartidos de forma segura en el entorno de la programación multihilo.

Podemos relacionar los elementos del problema con conceptos de programación multihilo:

- **Filósofo** → Hilo de ejecución independiente.
- **Tenedor** → Recurso compartido que no puede ser usado por más de un hilo a la vez.
- **Mesa y protocolo** → Estructura de sincronización.

Debemos hallar una solución que cumple con 3 garantías, los cuales son los objetivos de esta implementación:

1. **Seguridad:** Garantizar que dos filósofos no coman nunca a la vez con el mismo tenedor.
2. **Ausencia de interbloqueo:** El sistema no se queda bloqueado para siempre.
3. **Equidad:** Ningún filósofo puede quedarse sin comer de forma permanente.

2. Diseño de la solución

- **Explicación de la arquitectura general.**

La arquitectura general se basa en **3 componentes principales** que interactúan entre sí:

- **Filósofo (hilo activo)** → Su función lleva a cabo el ciclo: pensar, intentar comer, comer y dejar tenedores. El filósofo no toma los tenedores directamente, sino que solicita el permiso al monitor para asegurar que puede coger los tenedores si no están ocupados.
- **Tenedores (recursos)** → Representan el recurso compartido. Cada tenedor está asociado a un mecanismo de bloqueo que garantiza que, cuando un filósofo lo tiene, nadie más pueda acceder a él.
- **Monitor** → Actúa como el “administrador de la mesa”, es decir, el monitor de sincronización. Recibe solicitudes de los filósofos y decide si es seguro otorgarles los tenedores.

- **Descripción de las clases principales y sus responsabilidades.**

- **Filósofo** → Hilo que usa el monitor para poder ejecutar sus ciclos correspondientes.
- **Monitor** → Monitor de sincronización que gestiona y evita el interbloqueo. Asegura que todos los filósofos puedan coordinar sus ciclos de forma sincronizada y compartir los recursos (tenedores).
- **Cena de Filósofos** → Punto de entrada de la aplicación, crea N filósofos y el monitor compartido para coordinar el acceso a los tenedores.

- **Decisiones de diseño sobre el uso de monitores y/o semáforos.**

A. Uso de monitores:

El uso de un monitor garantiza que la **decisión sobre comer y asignar los dos tenedores al mismo tiempo se agrupe en una sola operación indivisible**, es decir, en vez de intentar coger un tenedor y luego otro (causando bloqueos), el monitor sincroniza que el filósofo coja ambos tenedores a la vez o espere a poder hacerlo.

B. Uso de semáforos:

El uso de un semáforo evita el **problema de interbloqueo** que puede surgir si los N filósofos se sientan y **cogen su tenedor izquierdo a la vez**. Actúa como un portero en la entrada del comedor, en el caso de haber 5 sillas solo dejaría entrar a 4 filósofos a la vez y, cuando uno de los filósofos coma, saldría y entraría al que espera fuera. Al haber solo 4 filósofos con 5 tenedores disponibles, se garantiza que al menos uno de ellos tendrá acceso a sus dos tenedores simultáneamente, rompiendo así el ciclo de interbloqueo.

3. Implementación concurrente

- Detalle del funcionamiento de **MonitorFilosofos**.

A. ESTADOS:

- ❖ **PENSANDO** → El filósofo no necesita recursos, por lo cual está tranquilo.
- ❖ **HAMBRIENTO** → El filósofo quiere comer e intenta conseguir los tenedores, pero no los tiene aún.
- ❖ **COMIENDO** → El filósofo tiene ambos tenedores y está comiendo.

B. INVARIANTE:

La invariante en este caso, es la condición en la que el **filósofo sólo puede pasar a estado “COMIENDO”** si se cumplen tres **cosas de forma simultánea**: que se encuentre en estado “HAMBRIENTO” y que su vecino de la izquierda y de la derecha no estén comiendo, garantizando de esta manera que dos vecinos nunca coman a la vez.

C. USO DE:

- ❖ **synchronized:**
 - Gracias a **synchronized**, métodos como **tomarTenedores()** y **dejarTenedores()** garantiza que sólo un filósofo a la vez pueda estar ejecutando código dentro del monitor. Si otro intenta entrar, debe esperar a que termine el anterior. De esta manera evita que dos hilos modifiquen el estado de la mesa al mismo tiempo y generen errores.
- ❖ **wait():**
 - Utilizado cuando un **filósofo quiere comer pero sus vecinos están comiendo** en ese momento.
 - Dentro de **tomarTenedores()**, si tras intentar comer no lo consigue, se ejecuta **wait()** para pausar el hilo y dejarlo dormido esperando el aviso de poder comer.
- ❖ **notifyAll():**
 - Se usa cuando un **filósofo termina de comer** en **dejarTenedores()**.
 - Después de soltar los cubiertos, llama al método para **despertar a los filósofos** que estaban dormidos **y esperando con wait()**, despertando así y volviendo a comprobar si sus vecinos han terminado de comer y puedan comer ellos.

- **Descripción de las variantes con semáforos y portero.**

- A. FilosofoConPortero:**

Esta variante, en su ciclo de vida, antes de intentar coger los tenedores, llama al método **sentarse()** para pedir **permiso al portero** (semáforo). Solo **cuando el portero le deja pasar**, coge los **cubiertos**, **come** y por último llama al método **levantarse()** para salir del comedor y **liberar su sitio** para otro filósofo.

- B. CenaFilosofosConPortero:**

El programa principal que prepara el escenario. Su tarea consiste en **inicializar el monitor** especial (MonitorFilosofoConPortero) **configurando la regla del aforo limitado**: si hay **N filósofos**, el semáforo sólo permitirá **sentarse a N - 1 a la vez**, asegurando dejar **siempre una silla libre**. También se encarga de **crear y lanzar los hilos** de los filósofos que usan este sistema.

- **Justificación de cómo se evita el *deadlock* y cómo se reduce la *starvation*.**

- ❖ **¿Cómo se evita el deadlock?**

Se evita gracias al **synchronized**, un filósofo verifica el estado de ambos vecinos (izquierda y derecha) al mismo tiempo en el método **intentarComer()**. Si ambos tenedores están libres, el filósofo pasa al estado **"COMIENDO"**, es decir, ocupa los dos tenedores. Si algún tenedor está ocupado, el filósofo se queda **esperando y duerme mediante wait()**.

Como nadie retiene un recurso mientras espera otro, **es imposible que se forme el interbloqueo**.

- ❖ **¿Cómo se reduce la starvation?**

Se reduce gracias a **notifyAll()** cuando el filósofo termina de comer y llama a **dejarTenedores()**, despertando a los compañeros dormidos para que intenten comer nuevamente.

Antes de pasar de estado, el filósofo **comprueba explícitamente si sus vecinos** (izquierda y derecha) estaban **esperando**, pudiendo comer en ese momento llamando a **intentarComer()**. Esto da **prioridad a los vecinos hambrientos** antes de que el monitor quede libre para otros.

4. Pruebas y resultados

- **Descripción de las pruebas realizadas.**

- ❖ **Escenarios de ejecución:**

Se simula una **mesa redonda donde los hilos compiten por recursos**. Se usan herramientas de sincronización como **CountDownLatch** para forzar que varios filósofos intenten comer exactamente al mismo tiempo, poniendo a prueba la **seguridad del Monitor**.

- ❖ **Número de filósofos:**

Las pruebas se han configurado con **5 filósofos** ($N = 5$).

- ❖ **Tiempos y Repeticiones:**

Se utilizan **esperas controladas** (de 20ms a 500ms) con **Thread.sleep()** para dar tiempo a que los hilos cambien de estado, asegurando que las operaciones de **wait()** y **notifyAll()** funcionan antes de verificar los resultados.

- **Resumen de las pruebas unitarias implementadas con JUnit.**

- ❖ **Pruebas de Inicialización y Estado:**

- **Caso:** Verificar el inicio del sistema.
 - **Resultado:** Se confirma que todos los filósofos empiezan en estado **“PENSANDO”** y que todos los tenedores están libres al arrancar. También se valida que el último filósofo tiene como vecino derecho al primero.

- ❖ **Pruebas de Exclusión Mutua:**

- **Caso:** Dos vecinos (Filósofo 0 y Filósofo 1) intentan comer simultáneamente.
 - **Invariante comprobada:** `assertFalse(f0Come && f1Come)`.
 - **Resultado:** Confirma que nunca es posible que dos vecinos estén en estado **“COMIENDO”** a la vez.

- ❖ **Pruebas de progreso:**

- **Caso:** Un filósofo termina de comer y libera los tenedores.
 - **Invariante comprobada:** El vecino que estaba esperando debe detectar los tenedores libres.
 - **Resultado:** El sistema actualiza el estado de los tenedores a **“libre”** y avisa a los vecinos que estaban esperando, permitiéndoles pasar a estado **“COMIENDO”**.

❖ **Pruebas de Ausencia de Deadlock:**

- **Caso:** Todos los filósofos intentan tomar su tenedor izquierdo al mismo tiempo.
- **Invariante comprobada:** El sistema no se congela de forma indefinida.
- **Resultado:** Se verifica que al menos un filósofo logra coger el segundo tenedor, garantizando que el ciclo pueda continuar y el programa no se detenga eternamente.

❖ **Pruebas de Ausencia de Starvation:**

- **Caso:** Ejecución del sistema bajo alta demanda durante un tiempo prolongado.
- **Invariante comprobada:** tiempo_sin_comer < limite_maximo
- **Resultado:** Se confirma que todos los filósofos entren al estado “**COMIENDO**” y no se queden esperando para siempre mientras sus vecinos comen.

○ **Selección de capturas de pantalla del depurador.**

```
32
33  @Override
34  public void run() {
35      try {
36          while (!Thread.currentThread().isInterrupted()) {
37              pensar();
38              System.out.println("Filósofo " + id + " 😊 tiene HAMBRE.");
39              // Puede quedar en estado WAITING dentro del monitor
40              monitor.tomarTenedores(id);
41          }
42      }
43  }
44
45  public synchronized void tomarTenedores(final int idFilosofo) throws InterruptedException {
46      estado[idFilosofo] = HAMBRIENTO;
47      intentarComer(idFilosofo);
48
49      while (estado[idFilosofo] != COMIENDO) {
50          wait();
51      }
52  }
53
54  private void intentarComer(final int idFilosofo) {
55      if (estado[idFilosofo] == HAMBRIENTO &&
56          estado[vecinoIzquierdo(idFilosofo)] != COMIENDO &&
57          estado[vecinoDerecho(idFilosofo)] != COMIENDO) {
58          estado[idFilosofo] = COMIENDO;
59
60          ocuparTenedor(tenedorIzquierdo(idFilosofo), idFilosofo);
61          ocuparTenedor(tenedorDerecho(idFilosofo), idFilosofo);
62      }
63  }
```

Figura 1.0 - Puntos de ruptura



Figura 1.1 – Vista de depuración que muestra la creación de los hilos correspondientes a los 5 filósofos. El sistema se ha detenido antes de que el primer filósofo intente tomar los tenedores.

estado	int[5] (id=33)
[0]	1
[1]	2
[2]	1
[3]	2
[4]	1

Figura 1.2 – Array “estado”. Se confirma que dos vecinos nunca comen a la vez (valor 2). El valor 1 (HAMBRIENTO) muestra que el filósofo espera sin bloquear al resto, permitiendo que los que están separados coman al mismo tiempo.

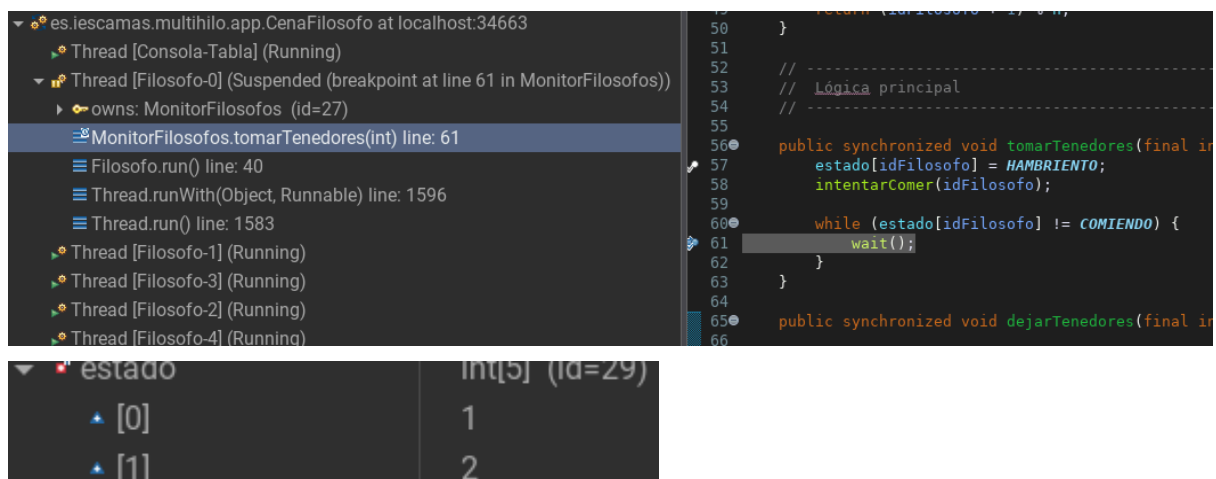


Figura 1.3 – El Filósofo 0 ejecuta wait() al detectar que su vecino (Filósofo 1) está COMIENDO. Esta acción libera el monitor temporalmente para evitar el deadlock, dejando al hilo en espera hasta que los tenedores queden libres.

5. Conclusiones y trabajo futuro

- **Resumen de lo aprendido sobre programación multihilo en Java.**
- ❖ **Concurrencia e Hilos:** Hemos aprendido cómo ejecutar múltiples tareas a la vez que comparten recursos limitados.
- ❖ **Monitores (synchronized):** Son esenciales para evitar interbloqueos e inanición. Actúan como un “cerrojo” que impide que dos hilos modifiquen los datos compartidos a la vez.
- ❖ **Coordinación (wait / notifyAll):**
 - **wait():** Libera el bloqueo mientras el hilo espera, permitiendo que otros entren al monitor, evitando el deadlock.
 - **notifyAll():** Despierta a los hilos dormidos para que comprueben que ya pueden actuar.
- ❖ **Problemas clásicos:**
 - **Deadlock (interbloqueo):** Cuando todos esperan a otro y nadie avanza.
 - **Inanición (starvation):** Cuando el sistema funciona, pero un hilo nunca recibe recursos.
- **Posibles mejoras o extensiones.**

Debido a la falta de profundización en otros temas como `ExecutorService`, mejoraría principalmente la aplicación con la creación de una **interfaz gráfica con Java Swing**, sustituyendo el texto de la consola por un `JFrame` con un dibujo de la mesa (un círculo central), con los 5 filósofos alrededor y mostrando sus distintos estados con colores, por ejemplo:

- **AZUL:** Pensando.
- **ROJO:** Hambriento.
- **VERDE:** Comiendo.